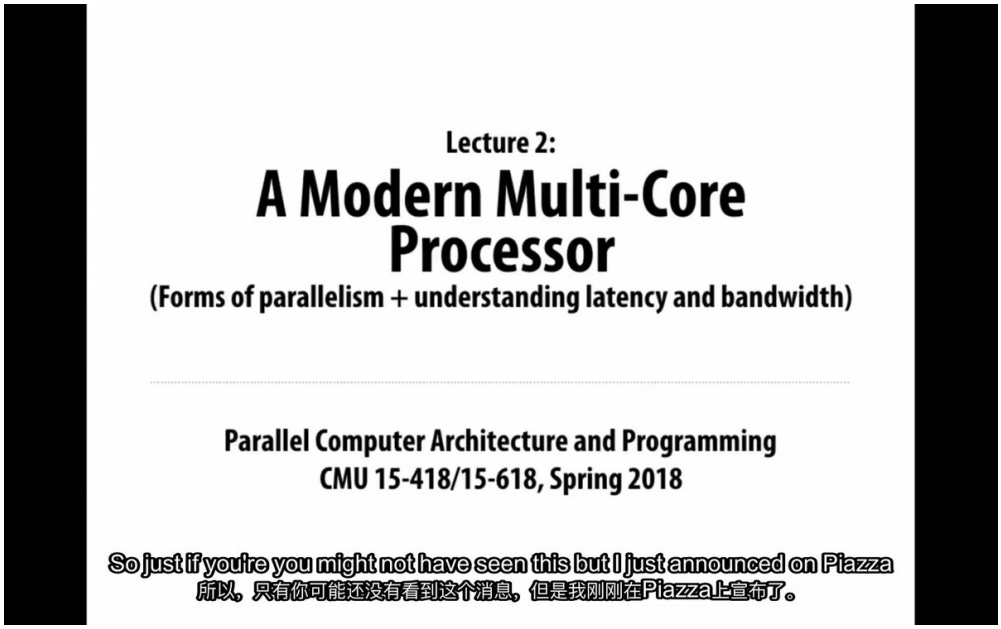
课程笔记

现代多核处理器：并行藏在哪些层次

CMU 15-418/618 并行计算机架构与编程 · Lecture 2（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 22 分 50 秒
视频链接：`https://www.bilibili.com/video/BV1Rh4y1F7aU/`

目录

1 硬件到底在哪些层次提供了并行 **3**
1.1 起点：功耗墙 . . . 3
1.2 并行藏在层次结构里 . . . 3
1.3 本章小结 . . . 3

2 最底层：指令级并行与顺序代码 **3**
2.1 一个没有并行的例子 . . . 3
2.2 超标量处理器怎么找并行 . . . 4
2.3 乱序执行：把指令流变成数据流 . . . 4
2.4 分支预测与它带来的安全问题 . . . 5
2.5 ILP 的天花板 . . . 5
2.6 本章小结 . . . 6

3 多核：拆了芯片，但程序还没准备好 **6**
3.1 0.75 倍的核心与 1.5 倍的潜力 . . . 6
3.2 纯顺序程序会变得更慢 . . . 7
3.3 用 pthreads 手工切分 . . . 7
3.4 更理想的表达方式 . . . 8
3.5 本章小结 . . . 9

4 SIMD：用一条指令驱动多个数据 **9**
4.1 纵向与横向的差别 . . . 9
4.2 AVX2：真实的向量指令 . . . 10
4.3 为什么不能指望编译器自动完成 . . . 11
4.4 条件分支：SIMD 效率的软肋 . . . 11
4.5 coherence 与 divergence . . . 12
4.6 向量还能有多宽 . . . 12
4.7 本章小结 . . . 12

5 GPU：把控制逻辑换成 ALU **13**
5.1 SPMD：对外是单程序多数据 . . . 13
5.2 去掉控制逻辑换来的算力 . . . 13
5.3 本章小结 . . . 15

6 内存：真正卡住并行的那一环 **15**
6.1 延迟的数量级落差 . . . 15
6.2 三级缓存与共享结构 . . . 15
6.3 预取：猜你要访问哪里 . . . 16
6.4 内存山：预取改变了曲线的形状 . . . 16
6.5 本章小结 . . . 18

7 多线程与超线程：用吞吐换延迟 **18**
7.1 执行上下文：多线程需要什么硬件 18
7.2 代价：单线程变慢 18
7.3 超线程的实现 19
7.4 GPU：把这条思路推到极致 19
7.5 本章小结 19

8 算术强度：峰值算力能兑现多少 **20**
8.1 思想实验：逐元素相乘 20
8.2 算一下这笔账 20
8.3 但即使饿着也快七倍 21
8.4 由此得到的判断规则 21
8.5 本章小结 21

9 总结与延伸 **21**

1　硬件到底在哪些层次提供了并行

上一讲用人类演示说明了并行会打折。这一讲换成硬件视角，回答一个更具体的问题：当程序员写下一行循环时，机器有哪些层次能同时干活，而每一层又要求程序员付出什么。讲者给出的答案是分层的——有些并行硬件自动完成，程序员看不见；有些必须由程序或编译器显式生成代码才能兑现。

1.1　起点：功耗墙

课程先回顾上一讲的结论。2004 年前后，芯片功耗密度触及上限：讲者给出的经验阈值是单芯片功耗超过约 100 瓦（视频 00:02:24）就难以散热。他用一个直白的类比说明这件事的严重性——想象把一个很亮的大灯泡塞进盒子里，你需要一台巨大的风扇。对手机尤其致命：电池会在五分钟内耗尽，而且机身会热到让人不适。

讲者的判断是人人知道这一天会来，却没人做好准备。转折之后，所有关于未来性能提升的预测都指向并行。

1.2　并行藏在层次结构里

这节课的主线可以概括为一句话：为了利用并行，硬件设计师在层次结构的多个层面上都布置了潜力，其中一部分名义上对程序员不可见，另一部分必须由程序员或编译器显式生成相应代码（视频 00:01:35）。

为什么”不可见”不等于”可以不管”

分支预测、乱序发射这类机制不需要写代码就能生效，但它们的效果高度依赖代码形态。同一份算法，写成什么形式决定了硬件能不能找到并行。讲者因此强调：必须把硬件理解到足以写出让硬件发挥潜力的软件。

讲者还点出贯穿全课的第二个主题：扩展计算规模时最大的难题往往不在计算，而在数据搬运。你可以有大量并行单元，但如果喂不进它们要的内存值，它们就没有用（视频 00:04:25）。这一讲的后半部分专门处理这件事。

1.3　本章小结

并行不是单一技术，而是分布在本章之后逐一展开的三到四个硬件层次上的能力；功耗墙是这一切的背景。既然层次不同，兑现方式也不同——有的全自动，有的要编译器配合，有的必须手写。下一章从最底层、也最”自动”的一层开始：从顺序代码中挖掘指令级并行。

2　最底层：指令级并行与顺序代码

即使程序是纯顺序的，硬件也在试图同时执行多条指令。这一层叫指令级并行（instruction level parallelism，ILP），理解它的上限，也就解释了为什么必须往上走到多核。

2.1　一个没有并行的例子

讲解用的例子是计算 sin 的级数项。核心汇编大致是三步：从内存把 x_i 读进寄存器、把寄存器自乘、再把结果与其中一个乘数相乘得到 x_i^3。讲者特意说明这份代码风格是”目的地写在左边”的

形式，与 2-13 课程里的 AT&T 格式相反。

关键在于依赖链：必须先完成读内存才能开始乘；必须先完成第一次乘才能做第二次乘。三条指令只能严格按序执行。任何并行机制在这里都找不到空隙。

依赖链决定可并行宽度

同一段代码能否并行，先看数据依赖，不看指令条数。读内存必须先于乘法，第一次乘法必须先于第二次，这三条指令因此只能串行。判断一份代码的并行潜力，第一步是把依赖图画出来，找出真正独立的操作有多少。

2.2 超标量处理器怎么找并行

从 1990 年代末开始，微处理器普遍具备超标量（superscalar）能力：硬件中有复制的逻辑从单一指令流中提取指令，并有多条 ALU 同时执行。

约束是语义保持——程序必须表现得像严格按顺序执行。硬件的工作是找出互不依赖的指令并同时发射。讲者提到，最初的 Pentium 大致就是这种形态。

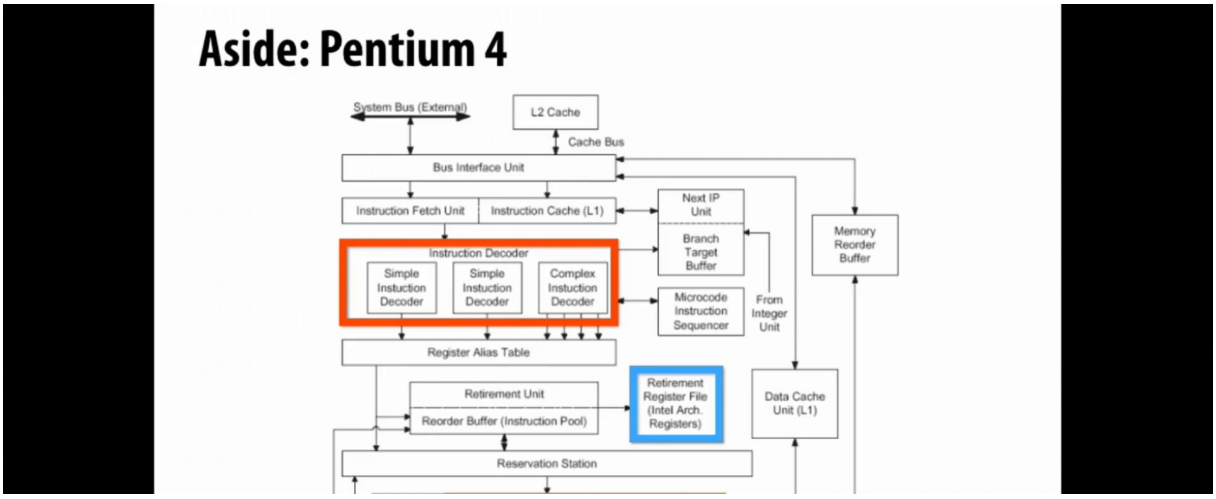


图 1: Pentium 4 的芯片结构：大片面积给了缓存与控制逻辑，ALU 只占相对小的一块[1]

2.3 乱序执行：把指令流变成数据流

1990 年代末之后，高端处理器转向乱序执行。做法是把程序计数器附近的约十条指令一并取出，映射成一种类似数据流的计算：跟踪哪个值会被哪条后续指令使用，也就是指令之间的数据依赖关系。然后通过复杂的控制逻辑，把这些操作分派到多个只能执行部分操作的独立单元上。

Pentium 4 的分工是：一个单元负责内存读写，两个做整数运算，一个做浮点运算，还有一个专门执行 SIMD 指令。较新的设计能力更强，可以同时两次加载一次存储，多个浮点与整数单元都能执行 SIMD 操作。

[1] 视频画面时间区间：00:10:45–00:11:00。若为裁剪图，时间区间仍对应原视频画面。

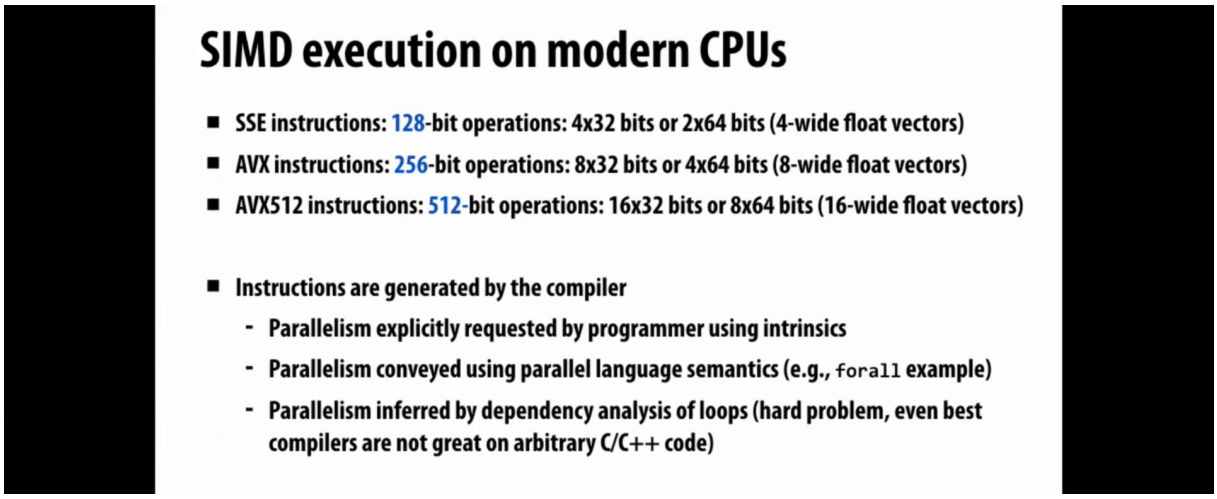


图 2: 现代 CPU 的 SIMD 能力：SSE 的 128 位与 AVX 的更宽向量[2]

讲者由此得出一个对顺序代码也成立的结论：即使完全串行的程序，运行快慢也取决于代码怎么写以及编译器有多聪明。硬件的原始算力就摆在那里，问题是有没有足够多的活喂给它。

乱序执行是硬件自动做的，但依赖代码形态

乱序执行、寄存器重命名、多发射都不需要程序员改代码就能生效，这是它们与 SIMD、多线程的根本区别。但硬件只能在指令窗口内寻找依赖关系，窗口之外无法利用。因此把独立计算写在一起、避免长依赖链，仍是程序员能直接影响这类加速的手段。

2.4 分支预测与它带来的安全问题

控制逻辑中有一块叫分支目标缓冲区，记录所有控制流指令过去跳向何处，据此预测将来也跳向同一目标。这样处理器不必等条件判断完成就能沿预判路径继续执行。

预测正确时一切顺畅；预测失败时，处理器不会把结果写回真实寄存器，而是直接冲刷掉，当作这段时间被浪费。

讲者在这里插入了一段当时正值新闻的旁白，值得单独记录：如果错误预测路径上访问了本进程无权访问的内存，虽然访问最终会被标记为失败，但缓存命中与否的差异会透露信息。这类时序与分支预测状态的泄露，正是 Spectre 一类攻击的原理。讲者强调这些机制位于系统最性能敏感的核心部位，因此堵住这类泄露很难。这也是为什么必须写清楚：这是讲者在课上给出的解释，不是本讲义补充的外部材料。

为性能而生的机制会成为攻击面

分支预测、乱序执行、预取都在缩短”看起来没用”的时间。一旦这些时间差可被观察，它们就成了信息泄露的信道。做安全敏感设计时，性能优化与信息隔离常常正面冲突。

2.5 ILP 的天花板

讲者给出这一层的量化上限：硬件能够从顺序代码中榨出的指令级并行大约是三到五倍（视频 00:19:17）。原因有两方面。其一，程序本来就不是按并行思路写的，指令之间的依赖限制了可同时

[2]视频画面时间区间：00:46:00–00:46:15。若为裁剪图，时间区间仍对应原视频画面。

执行的数量。其二，靠缩小晶体管提高主频这条路已经撞上功耗限制。

这一段是整讲的转折点：

$$S_{\mathrm{ILP}} \approx 3 \sim 5 \text{ 倍}, \qquad \text{与程序员是否改写代码无关}$$

- S_{ILP}：仅依靠超标量与乱序执行可获得的加速倍数；
- 该倍数由硬件对顺序代码的自动分析能力决定，不随处理器代际持续增长。

2.6 本章小结

指令级并行是硬件自动完成的一层，程序员无需改代码即可受益，但收益封顶在约三到五倍，且已在 2004 年前后用到尽头。要继续提速，必须让硬件同时做更多份互相独立的工作——这就需要把并行显式地暴露出来。下一章讨论由此产生的多核取舍。

3 多核：拆了芯片，但程序还没准备好

把一块大处理器拆成若干较小的处理器，是功耗墙之后的直接反应。这一章的重点不是多核本身，而是它暴露出的软件落差。

3.1 0.75 倍的核心与 1.5 倍的潜力

讲者设置了一个具体假设：假设每个新核心比原来的单核慢 25%，即性能是原来的 0.75 倍。那么放两个核心，理论上可望得到：

$$S(2) = 2 \times 0.75 = 1.5$$

- 0.75：每个新核心相对原单核的性能比例；
- 2：核心数量；
- 1.5：理想情况下两核并行能获得的加速倍数。

1.5 大于 1，看上去是赚了。但这只是硬件的潜力，兑现的前提是程序真的有这么多并行。

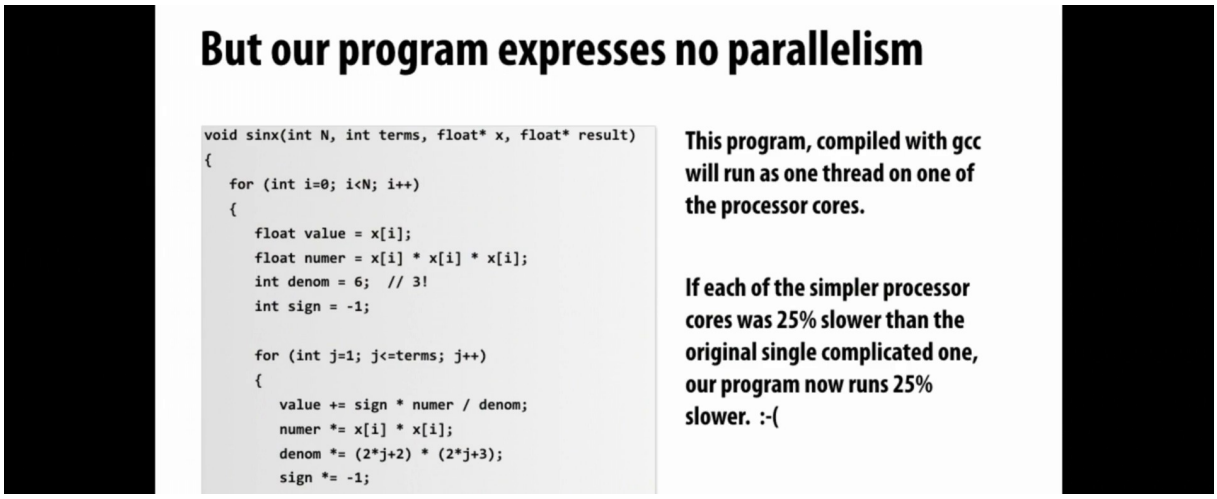


图 3: 双核并行计算两个元素的示意图，配页文字指出原程序本身不含并行[3]

3.2 纯顺序程序会变得更慢

讲者随即指出落差：如果一个纯顺序程序跑在这种双核机器上，它只会用到其中一个核心，于是只能拿到 0.75 倍——比原来的单核还慢。

这个结论的分量在于它把问题彻底翻转了过来。多核不是白送的性能，而是一次需要软件配合才能回本的交易；不配合时，硬件进步反而变成退步。

多核的红利是有条件的

$2 \times 0.75 = 1.5$ 只在程序能被切成两半并行执行时成立。纯顺序程序得到的是 0.75，比单核更差。核心数增加不自动等于性能增加，中间隔着并行化这一步。

3.3 用 pthreads 手工切分

第一条路径是手工写线程。讲者给出的做法很直接：把 n 个数分成两半，各约 $n/2$ 个，开一个线程算前半，主线程算后半。对于求这类计算，这个切分非常容易做对。

[3] 视频画面时间区间：00:20:45–00:21:00。若为裁剪图，时间区间仍对应原视频画面。

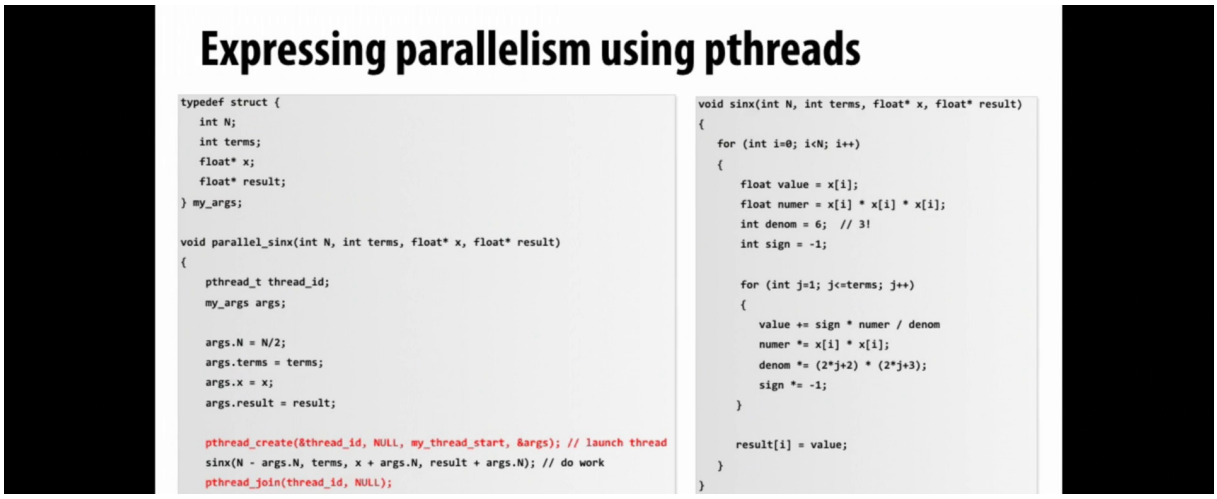


图 4: 用 pthreads 表达并行的代码：手工切分数据并创建线程[4]

但讲者对这条路的评价很坦率：pthreads 写起来不愉快，而更难的是在这套代码上做性能优化——很多地方拿不到预期的加速。这段评价呼应上一讲的三类代价：线程切分之后，负载均衡、通信同步、启动开销都仍然要程序员自己处理。

3.4 更理想的表达方式

讲者提出一个假想更好的世界：如果有语言能直接描述”工作项之间互相独立”这件事，就可以写成”对所有 0 到 $n-1$ 间的 i，执行如下操作”，把如何映射到硬件交给编译器与硬件去决定。

这种做法成立的理由是循环的每个迭代 i 都是完全独立纯粹的计算，不需要同步、协调或定序。讲者说作业一会用到具有这种风格的语言。

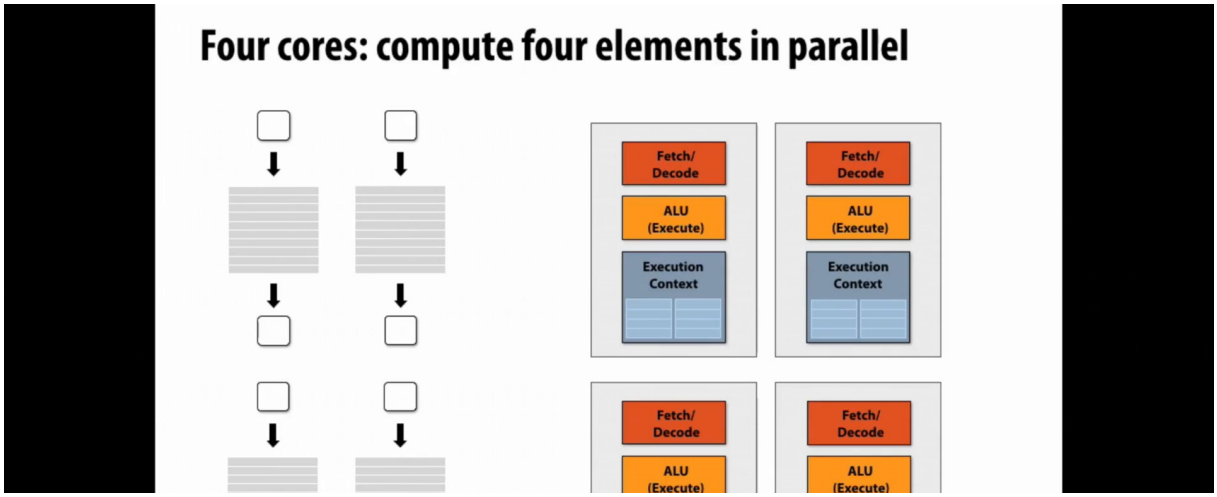


图 5: 四核并行计算四个元素，同一份计算在更多核上的映射[5]

[4] 视频画面时间区间：00:21:30–00:21:45。若为裁剪图，时间区间仍对应原视频画面。

[5] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

3.5 本章小结

多核把潜力写进了硬件，但兑现要求程序显式暴露并行。手工线程可以做到，代价是程序员承担切分、同步与调优的全部负担。既然”每个迭代独立”这种结构如此规整，硬件能否用一种更廉价的方式同时处理多个元素？这就是下一章 SIMD 的出发点。

4 SIMD：用一条指令驱动多个数据

多线程是纵向切分——每个线程负责一部分数据，各自跑完整个流程。SIMD 是横向切分：在同一时刻，对所有数据执行同一个步骤。

4.1 纵向与横向的差别

讲者用计算 x^3 的循环来说明这两种视角。线程视角是开 k 个线程，每个线程取一段 i 从头算到尾。SIMD 视角是同时对全部 i 执行第一次乘法，再同时对全部 i 执行第二次乘法，以此类推向上走。

要实现横向做法，需要往处理器里放更多 ALU，并假设所操作的运算彼此没有依赖，然后用一条指令把这一串操作串起来。这就是单指令多数据（single instruction multiple data，SIMD）。

讲者估算，16 个核心、每核 8 路向量处理，理论上能得到 128 倍的并行度，而代价相对很低，因为这些 ALU 占据的芯片面积很小（视频 00:36:48）。但这要求同时存在 128 件彼此完全独立的工作。

纵向切分与横向切分的适用条件

多线程（纵向）要求工作能切成独立的任务块，块之间几乎不通信；SIMD（横向）要求同一时刻各通道执行完全相同的操作。前者适合任务粒度大且彼此独立的场景，后者适合对整齐数组做同一变换的场景。两者可以叠加，但叠加后的实际并行度受较弱的那个条件约束。

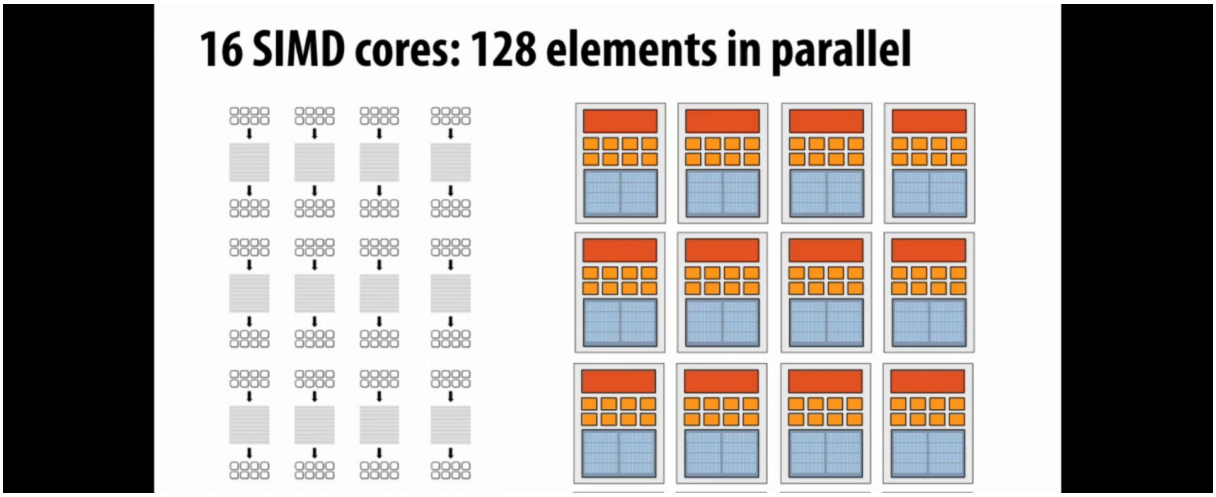


图 6: 16 个 SIMD 核并行处理 128 个元素：横向切分在多核上的叠加[6]

[6] 视频画面时间区间：00:36:55–00:37:10。若为裁剪图，时间区间仍对应原视频画面。

4.2 AVX2：真实的向量指令

Intel 当前的向量扩展叫 AVX2，即高级向量扩展第二代。它一次处理 32 字节，也就是 256 位（视频 00:33:01）；若数据是单精度浮点（每个 4 字节），则一次处理 8 个。

这套寄存器（XMM）与普通通用寄存器不同：通用寄存器只有 64 位，而 XMM 是独立的 256 位寄存器组，专为向量指令服务。现代处理器里即便非向量化的浮点运算也使用 XMM，但只用低位的几个字节。

调用这些指令的方式是 intrinsics：仍然写 C 代码，但实质是在逐条指示编译器该生成哪些汇编。命名中的 mm256 指 256 位；数据类型如 ___m256 就是 32 字节。ps 表示 packed single，即打包的单精度浮点。

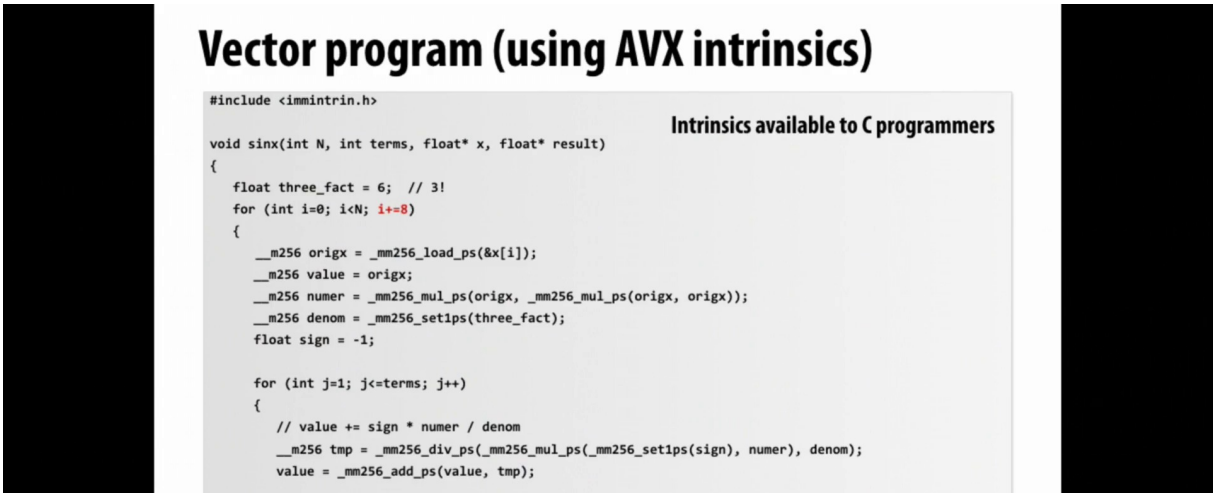


图 7：使用 AVX intrinsics 的向量程序：循环按 8 展开并逐批加载向量[7]

按讲者的描述，这段代码的效果是把循环按 8 展开，依次对前八个数据发出向量指令，再对下一批发出向量指令。生成的汇编与原版差别极大——程序员已经明确告诉处理器该如何取出八路并行。

intrinsics 把移植成本转嫁给程序员

用 intrinsics 写的代码与具体指令集绑定，AVX2 的 256 位写法不能直接搬到 SSE 或 ARM NEON 上，换架构就要重写。同时它牺牲了可读性：代码表达的是硬件动作，不是算法意图。讲者说这是”痛苦且不愉快的体验”，正是这个代价。

```
#include <immintrin.h>

for (int i = 0; i < N; i += 8) {
    // 一次载入 8 个单精度浮点 (32 字节 = 256 位)
    __m256 x = _mm256_load_ps(&A[i]);
    // 向量乘：x 自乘，得到 8 个 x_i^2
    __m256 x2 = _mm256_mul_ps(x, x);
    // 向量乘：再乘一次 x，得到 8 个 x_i^3
    __m256 x3 = _mm256_mul_ps(x2, x);
    _mm256_store_ps(&result[i], x3);
```

[7]视频画面时间区间：00:31:40–00:31:55。若为裁剪图，时间区间仍对应原视频画面。

```
11 }
```

Listing 1: AVX intrinsics 的核心写法（按讲者对 ___m256 与 _mm256_load_ps 的描述整理）

4.3 为什么不能指望编译器自动完成

有人问能否让编译器自动向量化。讲者的回答是” 长答案是能”，但需要给 GCC 大量提示与特殊条件，代码完全规整时才会触发。业界的共识是只有高度结构化、精心编写的代码能被自动向量化。

讲者给出的可靠验证手段很朴素：查看生成的.s 汇编文件，确认代码到底有没有被向量化。

4.4 条件分支：SIMD 效率的软肋

前面所有例子的共同优势是所有 i 执行完全相同的操作，因此容易向量化。一旦代码中出现依据 x 取值分支的情况（例如 $x > 0$ 时做一套操作、否则做另一套），问题就复杂了。

SIMD 的处理办法是掩码。对八个 x 并行做 $x > 0$ 的测试，得到一组真假值，把它当作掩码决定后续哪些操作生效：为真的通道执行，为假的通道被禁用，或虽然计算但结果不写回寄存器。然后翻转掩码，执行 else 分支的操作。

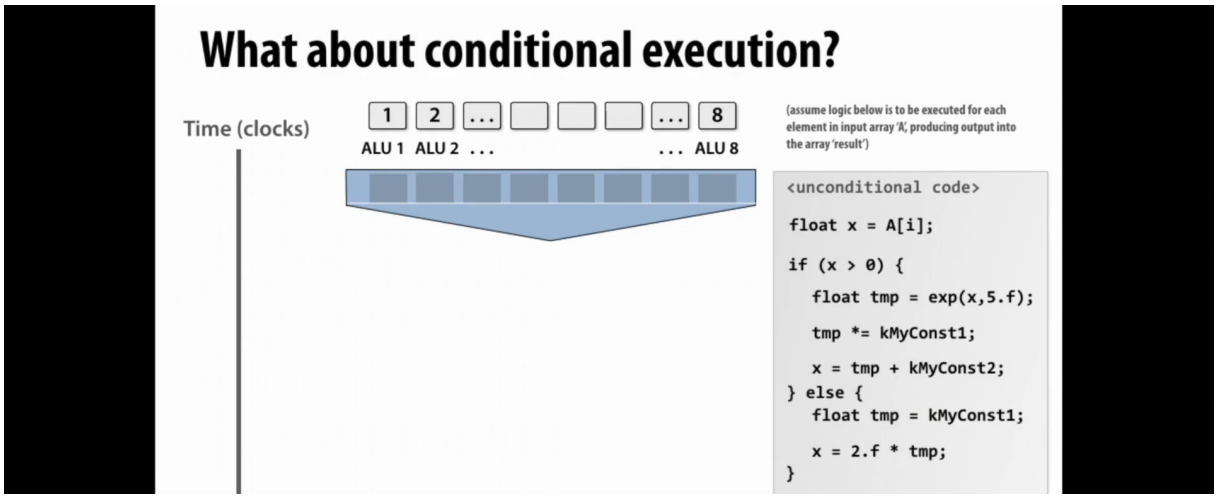


图 8: 条件执行的困境：八个通道里部分走 if、部分走 else[8]

这样做的代价是：分支被展平为直线代码，但两边的操作都要执行。若两路分支的工作量大致相当，那么任一时刻约有一半的 ALU 空转（即整体效率上限约 50%），效率最高只有 50%（视频 00:42:13）。极端情况下只有一两个元素真正需要执行，其余七个通道的算力全部浪费。

[8]视频画面时间区间：00:40:00–00:40:15。若为裁剪图，时间区间仍对应原视频画面。

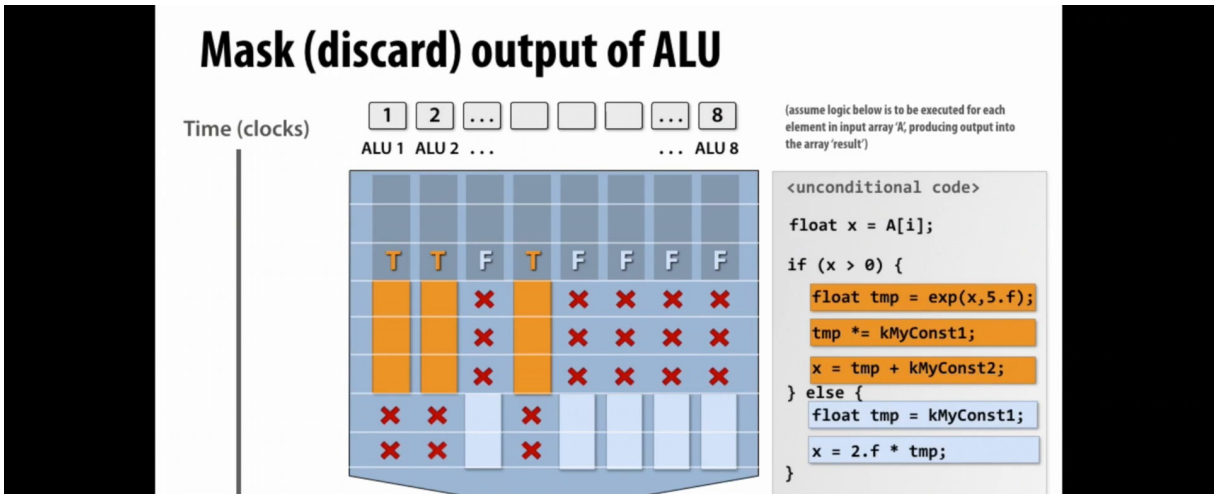


图 9: 用掩码丢弃 ALU 输出：为假的通道计算但不写回结果[9]

讲者也指出了好消息：分支结束后所有通道重新汇合到同一程序点，可以继续同步执行。

分支越细碎，向量化收益越低

掩码机制的成本取决于两路分支是否都要执行。若整批数据的判断结果一致，可以跳过另一路，效率接近满额；若两路混合，则两路都跑，效率腰斩；若只有个别通道需要走特殊路径，其余七路陪着空转，效率只剩八分之一。因此在 SIMD 代码里，把判断按数据分组往往比逐元素判断更快。

4.5 coherence 与 divergence

讲者为这一现象引入了两个术语，并特意提醒它们与后面讲缓存一致性时的 coherence 毫无关系。

coherence 与 divergence（SIMD 语境）

coherence 指同一组通道在多大程度上想做同一件事，程度越高，这批并行资源的利用率越高。divergence 指偏离这一理想的程度——通道之间的行为越不一致，被浪费的算力越多。判断一段代码是否适合 SIMD，很大程度上就是判断它会不会产生大量 divergence。

4.6 向量还能有多宽

讲者提到下一代是 AVX512，512 位。这里有一个命名上的提醒：512 是位数，与第几代无关，所以没有 AVX3。

关于是否继续加宽，他转述了内行的看法：这大概就是极限了。理由正是 divergence——真实程序里能够整齐一致的向量长度有限，一味加宽会让浪费的比例越来越高。

4.7 本章小结

SIMD 用一条指令驱动多个数据，在数据整齐时能廉价地成倍提高吞吐，16 核乘 8 路就是 128 倍。但它对数据一致性有强要求：一旦出现分支，掩码机制会让两路都执行，常态效率掉到一半，极

[9] 视频画面时间区间：00:41:40–00:41:55。若为裁剪图，时间区间仍对应原视频画面。

端情况只剩八分之一。这个弱点在 GPU 上被推向极致——下一章看 GPU 如何接受这个代价。

5 GPU：把控制逻辑换成 ALU

GPU 是 SIMD 思路的极端化。理解它的关键不是” 核多”，而是它主动放弃了什么。

5.1 SPMD：对外是单程序多数据

GPU 给程序员呈现的模型叫 SPMD（single program multiple data，单程序多数据）：不同部分可以执行不同的代码路径。但在底层，它仍然用 SIMD 来实现大部分逻辑。

程序员表达的是” 我要对两个各含 n 个值的向量做 n 次加法”，并可以启用或禁用其中某些通道；硬件则负责把这项工作切成大小固定为 32 个值（不是字节）的块，逐块执行。这个块大小叫 SIMD 宽度，历史上在 8 到 32 之间，目前多数是 32。

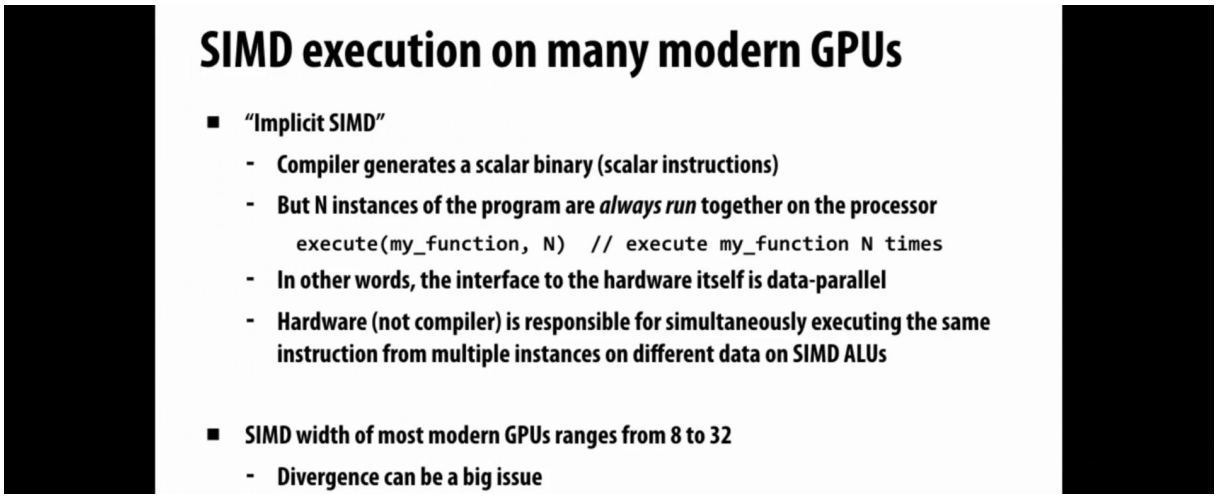


图 10: 现代 GPU 的隐式 SIMD：编译器生成标量二进制，硬件按 SIMD 宽度分块执行[10]

5.2 去掉控制逻辑换来的算力

GPU 的设计哲学可以用讲者一句话概括：分支预测、乱序控制这些逻辑是” 让你算不完的更多办法”，它们并不做程序真正要求的计算。GPU 把这些统统扔掉，塞进尽可能多的核心，用另一种特殊方式控制。

[10] 视频画面时间区间：00:49:20–00:49:35。若为裁剪图，时间区间仍对应原视频画面。

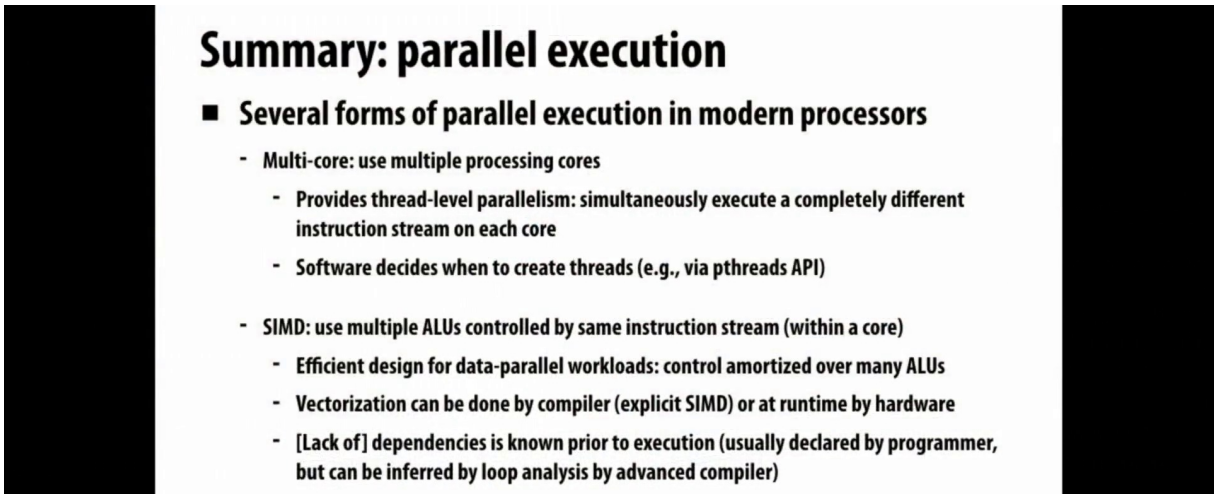


图 11: 并行执行方式汇总：多核、SIMD 与超标量三条路径[11]

讲者给出的真实机器参数让这个取舍变得具体。课程用的 Gates 机器是 2016 年前后的型号，4 个核心、每核支持 AVX2，按 4 字节数据算相当于每核 8 个并行 ALU。课程后期还会用到一台 12 核的集群。Gates 机器上还挂着 NVIDIA GPU，20 个核心、每核 32 个 ALU。

设备	核心数	每核 ALU	原始算力上限
Intel CPU（Gates）	4	8（AVX2）	不足 100 GFLOPS
集群节点	12	8（AVX2）	同量级
NVIDIA GPU（Gates）	20	32	8 TFLOPS

表 1: 课程所用机器的核心数、每核算力与峰值算力对照

算力差距是最有说服力的一点：GPU 全部加总能到 8 TFLOPS，即每秒八万亿次浮点运算；而 CPU 每核约二十几 GFLOPS，乘四核也不到 100 GFLOPS。两者之比约为 80 比 1（视频 00:52:32）。这个差距完全来自把控制逻辑的面积换成了 ALU。

上面这张表里 CPU 一列的数字来自讲者现场展示的 i7 参数页（视频 00:50:00）；GPU 一列取自同一实验室在用的显卡参数页，下图即该页。

[11] 视频画面时间区间：00:53:00–00:53:15。若为裁剪图，时间区间仍对应原视频画面。

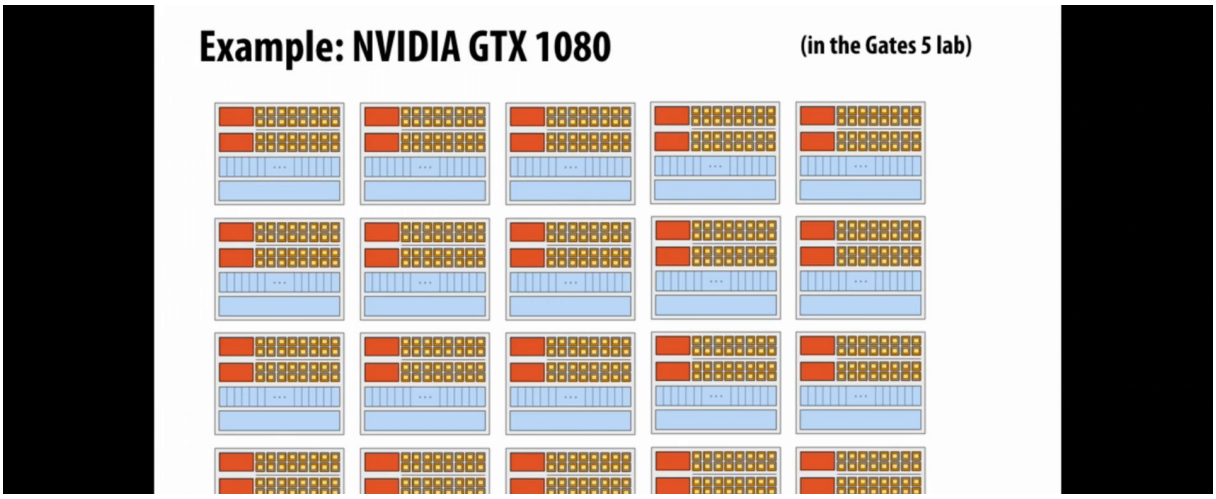


图 12: NVIDIA GTX 1080 的参数页：核心数与每核 SIMD 宽度的来源[12]

5.3 本章小结

GPU 用 SPMD 包装 SIMD，用放弃分支预测与乱序控制换来数十倍的 ALU 密度，因此峰值算力可达 CPU 的八十倍。但讲者留下一句关键限定：问题在于这些算力能不能被程序用起来。这句话的答案藏在内存里——下一章处理。

6 内存：真正卡住并行的那一环

计算能力已经很多了。这一章说明为什么大多数程序拿不到这些能力。

6.1 延迟的数量级落差

讲者指出，这些年芯片变快了、内存也变大了，但把一字节数据从内存送进处理器所需的时间几乎没有改进。这包含两个量：延迟，即一次访问需要多久；吞吐，即那条连接内存与处理器的相对狭窄通路上每秒能过多少字节。

两者的数量级值得记住。DRAM 访问时间约为 100 纳秒，而 1 GHz 处理器的时钟周期是 1 纳秒，现代处理器普遍在数 GHz（视频 00:54:38）。也就是说一次主存访问要等上几百个时钟周期。带宽则是每秒 25 GB/s 的量级（讲者说的数十 GB/s 按课程机器约 25 GB/s 计）。

6.2 三级缓存与共享结构

缓存的目的是让大多数访问落在更小更快的存储里，从而把 100 纳秒的延迟换成几个纳秒（讲者提到的量级是约 5 纳秒，视频 00:56:19）。典型机器有三级缓存。

多核机器上，靠近核心的缓存通常为每个核心私有，而最大的 L3 由所有核心共享。这一点有实际意义：核心之间可以通过一个写、另一个读来传递数据，这些数据只需经过 L3，不必一路写到主存。

[12] 视频画面时间区间：00:51:20–00:51:35。若为裁剪图，时间区间仍对应原视频画面。

缓存解决延迟，带宽解决不了

缓存把常见访问从主存搬到片上，改善的是单次访问要等多久。它同时也能提高有效带宽，因为片上连线短且数量多。但这两种收益都有上限：缓存容不下工作集时，访问仍会落到主存，此时决定性能的是内存总线的每秒字节数，与缓存层级设计无关。

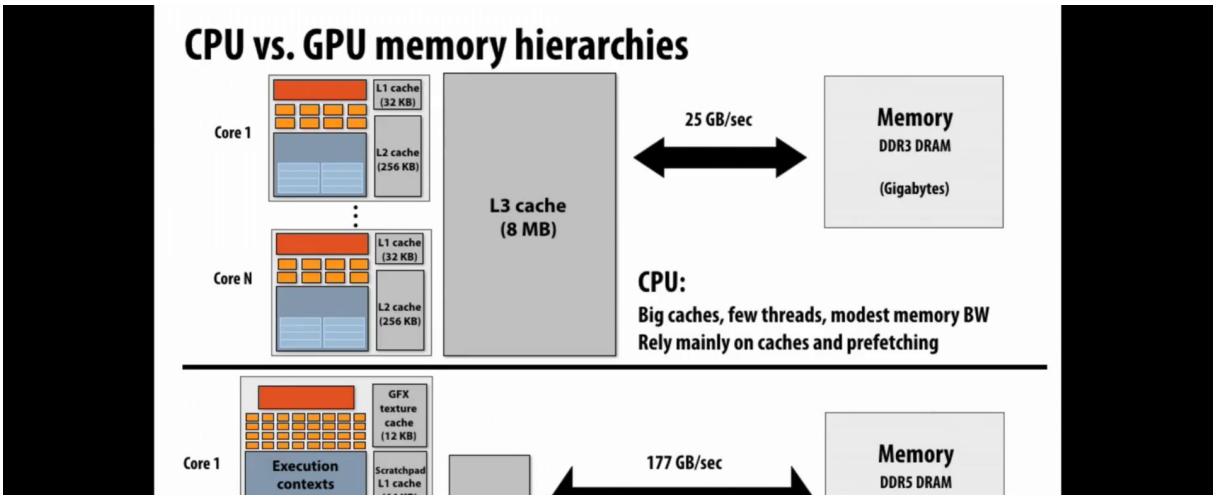


图 13: CPU 与 GPU 的内存层次对比：缓存容量、私有与共享关系、以及带宽标注[13]

6.3 预取：猜你要访问哪里

既然延迟这么高，硬件就像猜分支方向一样去猜你接下来要访问哪个内存地址，这就是预取。

它并不读你的代码，只观察地址序列：如果发现你在按固定步长（例如每隔四个位置）访问，就推断这个模式会继续，并提前把数据从主存搬进缓存。这样等你真正执行到那条加载指令时，数据已经在缓存里了。现在几乎所有硬件都做这件事。

6.4 内存山：预取改变了曲线的形状

讲者用两台机器的对比来说明预取的效果。他用家里一台 2008 年的 iMac 测过一组数据，那是当年相当不错的处理器；另一组是课程用的 Gates 机器。

横轴是数组总大小，从左端每次只访问一个字开始向右增大。左端数据量小，全部在 L1 里反复循环，带宽最高。越过 L1 容量后开始不断缺失（miss），带宽出现明显下跌，随后数据稳定落在 L2 中，这个台阶反映的是 L1 与 L2 的相对性能。再往右落入主存，又一次下跌。

[13] 视频画面时间区间：00:48:00–00:48:15。若为裁剪图，时间区间仍对应原视频画面。

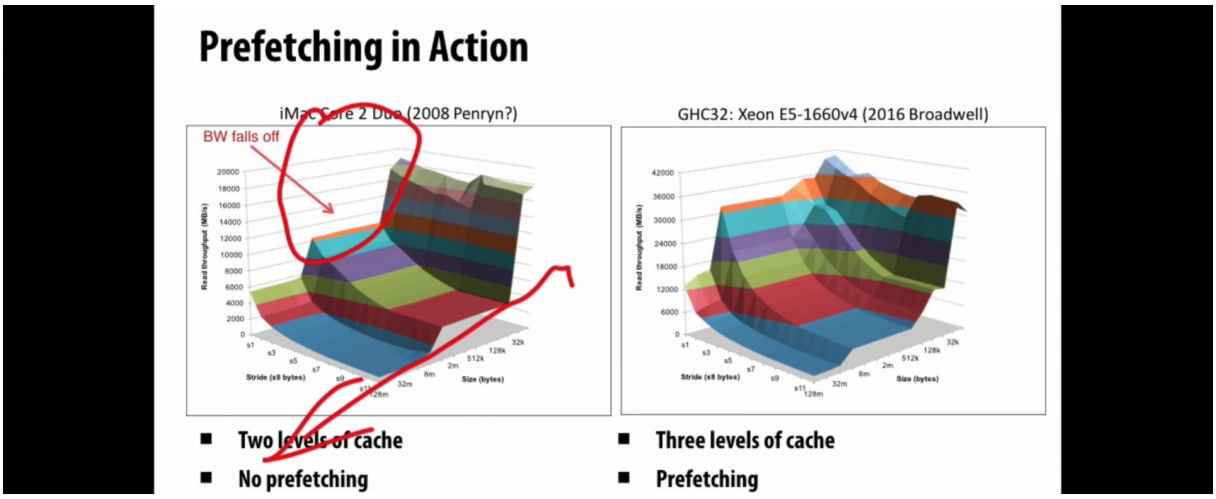


图 14: 预取生效的实证：新一代机器的带宽台阶更平缓，且出现 L3 的额外凸起[14]

Gates 机器的曲线差异极大：下跌幅度小得多，中间还多出一个小凸起，那是 L2，后面是 L3。讲者明确把这一改善归因于预取。

但预取的作用有严格边界：它能减少延迟，却不能提高吞吐。一旦访问规模超出 L3，仍然要撞上内存总线有限的带宽，此时曲线再次陡降。讲者说他跑到树莓派之类的设备上才容易观察到没有预取的原始形态。

预取对随机访问无效

硬件预取依赖可识别的地址模式，例如固定步长的循环。指针追逐、哈希查找这类访问模式没有规律，预取器无法预测，延迟会完整暴露。这类代码的优化方向是改变数据布局以提高局部性，而不是指望硬件猜准。

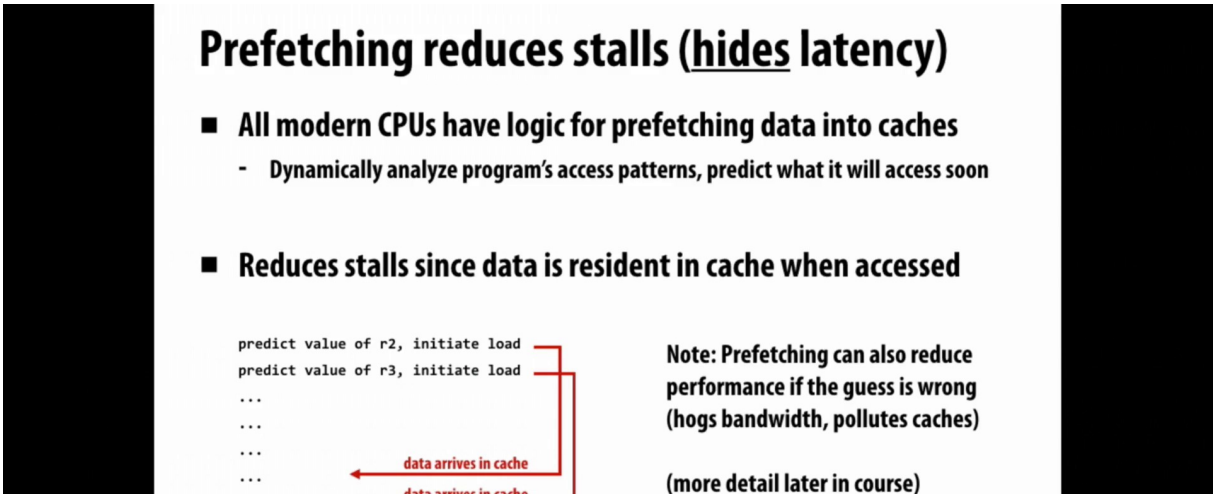


图 15: 预取的机制：根据地址访问模式提前把数据搬入缓存[15]

[14] 视频画面时间区间：01:00:00–01:00:15。若为裁剪图，时间区间仍对应原视频画面。

[15] 视频画面时间区间：00:58:00–00:58:15。若为裁剪图，时间区间仍对应原视频画面。

6.5 本章小结

主存延迟比时钟周期高两个数量级，缓存把多数访问拉回纳秒级，预取通过猜测地址模式进一步压缩等待。但预取只改善延迟，不改变带宽，一旦数据量超过共享缓存，性能就由内存总线决定。这条限制指向一个尚未使用的手段：既然处理器在等内存时闲着，能不能让它去干别的？

7 多线程与超线程：用吞吐换延迟

这一章回答上一章末尾的问题。核心思路是：等待内存时处理器无事可做，那就安排另一条独立的执行流在这些空隙里运行。

7.1 执行上下文：多线程需要什么硬件

做法是把多个独立线程的处理交错在同一核心上。设想第一个线程发起一次八路向量加载，数据不在缓存里，它就停顿了。如果此时另一个线程也有八次读取准备好，硬件就可以发起它，让它在同一个停顿窗口里推进。

这需要的硬件不是更多 ALU，而是多个执行上下文。讲者给的定义是：每个线程的寄存器状态，加上足以存放栈或至少是栈的当前部分以及其他必要状态的内存。所有这些线程还会共享同一批缓存。

共享缓存带来一个真实的副作用：内存不足时线程之间会互相挤占缓存。但讲者的判断是这件事总体划算，因为它们填进去的正是本来无所事事的时间。

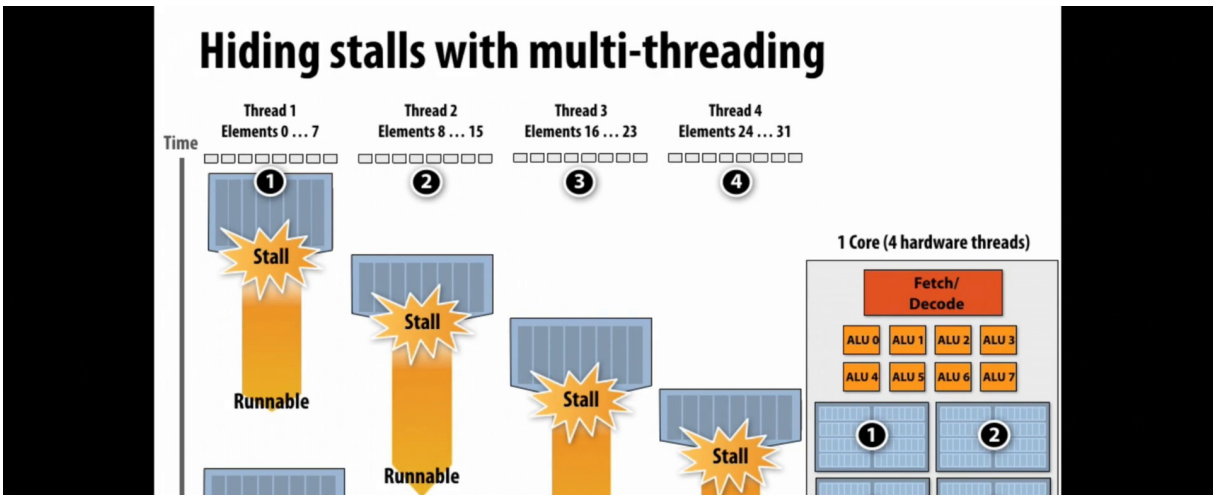


图 16: 用多线程隐藏停顿：四条线程在时间轴上交替填充彼此的等待空隙[16]

7.2 代价：单线程变慢

讲者把这类做法称为面向吞吐的计算。他坦率承认代价：上面的例子里，第一条线程的单独性能其实是下降的，因为有时别的线程会占用资源，即使它当时已经可以继续执行，也得等。

这引出了课程反复出现的一条主线：

$$\text{并行化} \Rightarrow \text{关键资源从延迟转向吞吐}$$

[16] 视频画面时间区间：01:04:00–01:04:15。若为裁剪图，时间区间仍对应原视频画面。

- 延迟：单次操作需要多长时间；
- 吞吐：单位时间内系统能送过多少工作；
- 讲者的判断是提高吞吐比降低延迟更容易实现。

代价的另一个侧面是资源分配：只跑一个线程时，整个 L1 缓存和寄存器组都可以给它用；如果要交错十六条线程，每个线程能分到的存储就更少。

7.3 超线程的实现

Intel 把这项技术叫超线程（hyperthreading），非商业的叫法是同时多线程（simultaneous multithreading）。讲者特意指出它并非 Intel 发明，所以这个名字不算他们的功劳。

实现方式是利用现代乱序处理器中本就存在的大量硬件：增加少许逻辑，并把寄存器复制若干份，让每个线程有独立的寄存器组，而执行单元、内存与缓存仍然共享。当某线程因内存问题停顿，硬件立即切换到另一线程执行。

7.4 GPU：把这条思路推到极致

在 GPU 上这种做法被推到极端。讲者以某个型号为例：每个核上相当于 48 路交错的线程，而这样的核有 15 个。相乘之后，需要有两万三千多个可独立操作的数据值，才能让整块芯片持续跑满。这个逻辑并发度极高。

代价同样清晰：GPU 的内存系统是为极高吞吐调优的，缓存相对较小，连主存容量也不能做得很大。缩小缓存腾出的面积，又被用来堆 ALU。

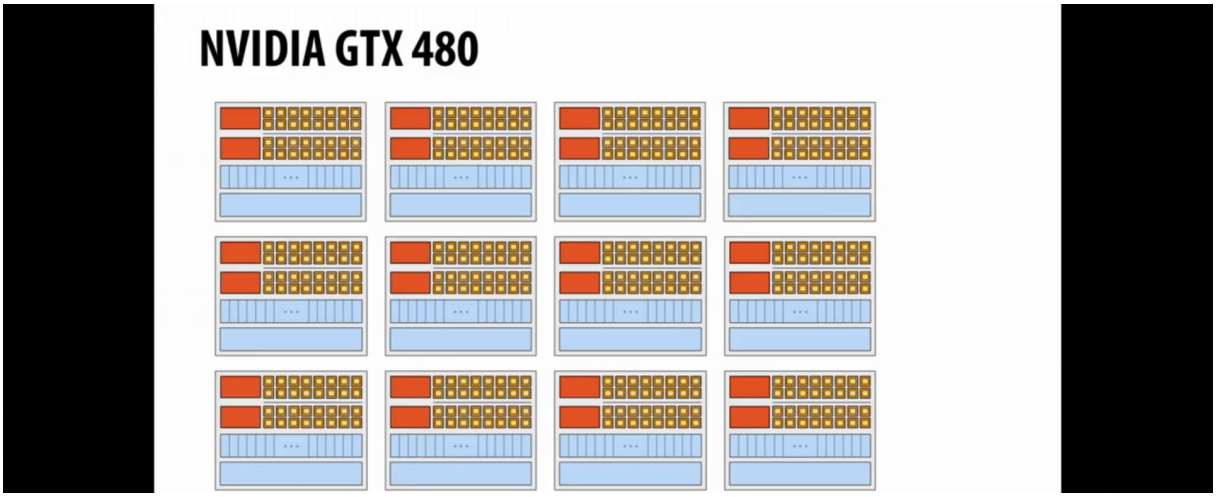


图 17: NVIDIA GTX 480 的结构细节：每核多路线程与宽向量单元[17]

7.5 本章小结

多线程用多个执行上下文填充内存等待的空隙，本质是把延迟问题转化为吞吐问题，代价是单线程性能与缓存份额。超线程让这件事在普通 CPU 上几乎免费；GPU 则把它与宽 SIMD 叠加，得到极高的逻辑并发度，同时换来小缓存与有限主存。至此所有提高算力的手段都已就位，最后要问的是：这些算力实际能用上多少？

[17] 视频画面时间区间：01:17:00–01:17:15。若为裁剪图，时间区间仍对应原视频画面。

8 算术强度：峰值算力能兑现多少

这一章用一次思想实验给出全讲最有说服力的定量结论。

8.1 思想实验：逐元素相乘

任务是对两个向量 A 和 B 做逐元素相乘，结果写入 C。向量包含数百万个元素。每个元素需要两次加载（A[i] 与 B[i]）、一次乘法、一次存储。

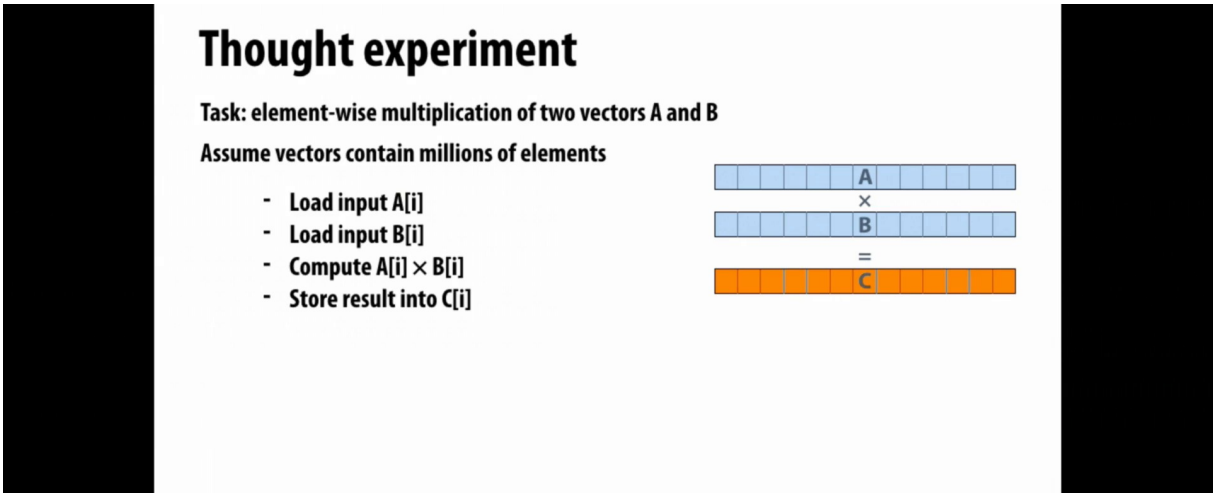


图 18: 算术强度思想实验：逐元素相乘每次运算需要两次读取与一次写入[18]

讲者指出这段代码性能表现很差，因为它对内存极度饥渴：每做一次算术运算就要两次读和一次写。这类代码的特征叫算术强度低。

8.2 算一下这笔账

讲者用某个型号的 GPU 做了具体计算。它每时钟周期能做 400 次乘法。要让这些乘法全部有数据可用，需要的带宽是：

$$B_{\text{need}} = \frac{400 \times 4\text{字节} \times 3\text{次访问}}{\text{每周期}} \approx 6.4 \text{ TB/s}$$

- 400：该 GPU 每时钟周期可完成的乘法次数；
- 4 字节：每个单精度浮点数的宽度；
- 3 次访问：每个元素两次读加一次写；
- 6.4 TB/s：喂满这些乘法所需的内存带宽。

该 GPU 实际只有 177 GB/s（视频 01:19:42），缺口超过三十倍。于是它只能被这条管道完全限制住：

$$E_{\text{mem}} = \frac{177}{6400} \approx 3\%$$

[18] 视频画面时间区间：01:19:00–01:19:15。若为裁剪图，时间区间仍对应原视频画面。

- 177 GB/s：该 GPU 的实际内存带宽；
- 6400 GB/s：充分供数所需带宽；
- 3%：实际能达到的算力占峰值的比例。

讲者的说法是这块核被内存带宽饿死了，效率只有 3%（视频 01:19:55）。

8.3 但即使饿着也快七倍

这个结论有个容易误读的地方，讲者专门澄清：即便被带宽卡死，这个 GPU 仍然比同代的 Xeon 快大约七倍（视频 01:20:00）。原因是 Xeon 的带宽更低（讲者提到约 25 GB/s，现代 Xeon 更高一些），在这种以带宽为主导的计算上同样受限，而且起点更差。

不要把 3% 误读为"GPU 不适合这类计算"

3% 的峰值效率听起来像失败，但它的比较对象是该 GPU 自己的算力峰值，不是 CPU。在算术强度低的计算里，GPU 的绝对速度仍可能是 CPU 的数倍。判断用哪个设备，要看绝对耗时，而不是各自峰值利用率的比例。

8.4 由此得到的判断规则

讲者把这条经验提升为课程的一条主线：内存速度往往比算术速度更是限制，无论你编的是 GPU 还是传统处理器。

场景	决定性能的资源	优化方向
算术强度高	算术单元	提高并行度、向量化、减少分支
算术强度低	内存带宽	提高数据复用、分块、减少搬运
数据超出共享缓存	内存总线带宽	改善访问局部性，而非增加核心

表 2: 按算术强度判断瓶颈所在的规则

8.5 本章小结

当每字节数据只参与很少几次运算时，算力规模毫无意义，性能由带宽决定：400 次乘法需要 6.4 TB/s，实际只有 177 GB/s，效率落到 3%。但这条管道劣势下 GPU 仍比 CPU 快七倍，说明比较必须用绝对速度。讲者的结论是本课最重要的判断依据——多数程序的瓶颈在供数，不在算力。

9 总结与延伸

这一讲没有写任何并行算法，它给出的是硬件的能力地图，以及读取这张地图的方法。

讲者开篇设下的主线在结尾闭合：为了利用并行，硬件在多个层次上都放置了潜力，其中一部分对程序员不可见，一部分必须显式生成代码。全讲把这些层次一一展开——指令级并行（自动，上限三到五倍）、SIMD（需要 intrinsics 或聪明的编译器，宽度从 128 位到 512 位）、多核（需要显式线程）、多线程与超线程（用吞吐换延迟）、GPU（放弃控制逻辑换八十倍 ALU 密度）。算力一路放大到 128 倍乃至更高，但每一层都附带一个兑现条件。

本讲义在讲者材料之上做的压缩，有四点值得单独记住，需与讲者原话区分：

- 算力的增长与可用性并不同步。峰值从不足 100 GFLOPS 涨到 8 TFLOPS，但低算术强度的核只用到 3%，实际收益取决于数据能否供上。
- SIMD 的效率由数据一致性决定。128 倍的潜力在两路分支下常态只剩一半，极端情况只剩八分之一；divergence 是判断代码是否适合向量化的核心指标。
- 延迟与吞吐是两个必须分开讨论的目标。缓存与预取处理延迟，多线程把延迟转成吞吐，而带宽是两者都无法突破的硬约束。
- 判断瓶颈要用绝对速度而非峰值利用率。3% 的效率仍可能比另一台机器快七倍。

这堂课的边界同样明确。讲者反复说明分支预测、乱序执行、超线程的细节更属于计算机体系结构课程，本课不会深挖；SIMD 与多线程的具体编程会在后续作业中展开。也就是说，本讲给出的是选型与诊断的框架，不是调优手法。

两个可以直接从本讲数据提出的下一步问题：其一，16 核乘 8 路的 128 倍潜力，在真实的分支代码里平均能兑现多少，这个折扣与第 3 讲的负载不均折扣相比谁更大？其二，如果算术强度是决定因素，那么把同一份计算分块以提高缓存复用，能使 3% 的效率提高到什么水平？二者的答案都取决于具体的数据规模与访问模式，也因此需要第 3 讲要讲的性能测量方法，而不是先验推断。